Práctica 3 y 4. Preparación y manejo de Datos

Análisis Exploratorio de Datos, Máster en Ciencia de Datos - UV

Contents

TidyR **2**

Introducción . . . 2

Librería `tidyr`: organización de datos en tablas . . . 3

Librería `magrittr`: ceci n'est pas une pipe . . . 4

Ejemplo de visualización . . . 5

Las cabeceras de las columnas son variables en lugar de nombres de variables . . . 5

Ejercicio 1 . . . 6

Ejercicio 2 . . . 9

Varias variables en una misma columna . . . 10

Ejercicio 3 . . . 11

Variables almacenadas tanto en filas como en columnas . . . 12

Ejercicio 4 . . . 12

DPLYR **13**

Introducción y objetivos. . . . 13

Ejercicio 5 . . . 14

Ejercicio 6 . . . 15

Ejercicio 7 . . . 16

Ejercicio 8 . . . 16

Ejercicio 9 . . . 16

Ejercicio 10 . . . 17

Ejercicio 11 . . . 18

Ejercicios Extra **18**

TidyR. Varias unidades de observación en una sola tabla . . . 18

Ejercicio Extra 1 . . . 18

TidyR. Una única unidad de observación almacenada en varias tablas . . . 20

TidyR. Otro conjunto messy... . . . 20

Ejercicio Extra 2 . . . 20

DPLYR . . . 21

Ejercicio Extra 3 . . . 21

Ejercicio Extra 4 . . . 22

Ejercicio Extra 5 . . . 23

Ejercicio Extra 6 . . . 23


```
library(pacman)
packages=c("tidyverse", "knitr", "ggplot2", "datasets", "RColorBrewer",'dplyr','lubridate',"nycflights13
pacman::p_load(char=packages)
```

TidyR

Introducción

El objetivo de esta práctica es preparar los datos de una manera estructurada. Para ello vamos a seguir las recomendaciones del artículo de Hadley Wickham, *Tidy Data*, donde nos muestra una forma de estructurar (u organizar) los datos, de manera que se faciliten posteriores cadenas de procesado (visualización, obtención de datos estadísticos, modelización, etc.).

Decimos que un conjunto de datos está estructurado u organizado (se usa el término *tidy* en inglés) cuando cumple estas tres condiciones:

1. Cada variable está en una columna.
2. Cada observación está en una fila.
3. Cada unidad de observación está en una tabla.

Por el contrario, si no se cumple cualquiera de estas tres condiciones decimos que los datos están desordenados (*messy*).

Parece fácil, pero el problema viene al definir exactamente qué es lo que entendemos por variables y observaciones.

- Una variable (columna) contiene los valores correspondientes a un atributo (ej. altura, tiempo, etc.).
- Una observación (fila) contiene los valores medidos sobre la misma unidad (una persona, una fecha, una muestra etc.) a lo largo de los atributos.

Vamos a ver un ejemplo con el dataset IRIS.

```
head(iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
```

¿Es un conjunto *tidy*?

- ¿Cada variable está en una columna? Depende de lo que definamos como variable. Según hemos visto, una variable debería contener todos los valores que miden un atributo. En este caso el atributo `Width` aparece en dos columnas, `Sepal.Width` y `Petal.Width`. Lo mismo ocurre con `Length`. También las partes de la flor: `Sepal` y `Petal` aparecen en varias columnas.
- ¿Cada fila es una única observación?

Veamos que aspecto tiene el conjunto IRIS una vez estructurado:

```
load('data/iris_tidy.Rdata')
head(iris.tidy)
```

```
  Species  Part Measure Value
1  setosa Sepal  Length   5.1
2  setosa Sepal  Length   4.9
3  setosa Sepal  Length   4.7
4  setosa Sepal  Length   4.6
5  setosa Sepal  Length   5.0
6  setosa Sepal  Length   5.4
```

Como se puede observar, en la versión *tidy* se han reorganizado los datos de manera que todos los valores medidos en una misma unidad ocupan una única columna. Para ello se han creado dos nuevas columnas correspondientes a dos nuevas variables, `Part`, que indica si se trata de `Sepal` o `Petal`, y `Measure`, que indica si se ha medido `Length` o `Width`.

Librería `tidyr`: organización de datos en tablas

Para ayudarnos en la tarea de estructurar los datos según este formato *tidy*, en R se dispone de la librería `tidyr`. Consulta cualquier cheatsheet sobre tidyR para ver cuáles son las funciones más importantes de esta librería. En esta práctica veremos sobretodo las siguientes:

- `gather(data, key, value, ...)`: permite agrupar o juntar varias columnas en una sola en función de una misma clave (`key`) que toma los valores de los encabezados de las columnas agrupadas, y una columna `value` toma los valores de sus celdas.
- `spread(data, key, value, ...)`: hace lo contrario que `gather`, divide o esparce los valores que toma una variable dada por `key`/`value` como nuevas columnas cuyos nombres son los valores de `key`.
- `separate(data, col, into, ...)`: separa los contenidos que hay en una columna en varias columnas sin cambiar el número de observaciones (no confundir con `spread`).
- `unite(data, col, ...)`: hace lo contrario de `separate`, junta los contenidos de varias columnas (no confundir con `gather`).

Aunque `gather` y `spread` se siguen utilizando, existen dos nuevas funciones de `tidyr` que tienen un comportamiento similar:

- `pivot_longer(data, cols, names_to, values_to)`: condensa la información de las columnas `cols` en dos, dadas por los valores de sus encabezados (`names_to`), y los contenidos de sus celdas (`values_to`).
- `pivot_wider(data, names_from, values_from)`: es el caso inverso: extiende las columnas dadas por `names_from` y `values_from` a tantas columnas como valores tome la variable `names_from`.

También son interesantes las siguientes:

- `drop_na(data, ...)`: elimina las filas que contienen NA's.
- `fill(data, ...)`: sustituye los NA's por el valor más reciente no NA.
- `replace_na(data, replace=list(), ...)`: reemplaza los NA's con los valores suministrados.

Volviendo al ejemplo del dataset IRIS, veamos cómo podemos usar `gather` y `separate` para convertir el conjunto original `iris` en el conjunto `iris.tidy`.

```
library(tidyr)
# Primero agrupamos todas las unidades
# Agrupamos todas las columnas con datos excepto Species, que está bien
# iris.t <- gather(data=iris, key="Part", value="Value", 1:4)
#iris.t <- gather(iris, Part, Value, Sepal.Length:Petal.Width)
iris.t <- gather(iris, Part, Value, -Species)
head(iris.t)
```

```
  Species         Part Value
1  setosa Sepal.Length   5.1
2  setosa Sepal.Length   4.9
3  setosa Sepal.Length   4.7
4  setosa Sepal.Length   4.6
5  setosa Sepal.Length   5.0
6  setosa Sepal.Length   5.4
```

```
# Luego separamos la variable Part en dos, Part y Measure
iris.t <- separate(iris.t, Part, into=c("Part", "Measure"))
head(iris.t)
```

```
  Species  Part Measure Value
```

```
1  setosa Sepal  Length   5.1
2  setosa Sepal  Length   4.9
3  setosa Sepal  Length   4.7
4  setosa Sepal  Length   4.6
5  setosa Sepal  Length   5.0
6  setosa Sepal  Length   5.4
```

Ahora ya tiene formato *tidy*. Cada columna contiene una variable, y cada fila una observación.

Podemos hacer lo mismo con `pivot_longer`. Observa como cambia la ordenación de las filas.

```
iris.t.2 <- pivot_longer(iris, col=-Species, names_to="Part", values_to="Value")
head(iris.t.2)
```

```
# A tibble: 6 x 3
  Species Part         Value
  <fct>   <chr>        <dbl>
1 setosa  Sepal.Length   5.1
2 setosa  Sepal.Width    3.5
3 setosa  Petal.Length   1.4
4 setosa  Petal.Width    0.2
5 setosa  Sepal.Length   4.9
6 setosa  Sepal.Width    3
```

```
# Luego separamos la variable Part en dos, Part y Measure
iris.t.2 <- separate(iris.t.2, Part, into=c("Part", "Measure"))
head(iris.t.2)
```

```
# A tibble: 6 x 4
  Species Part  Measure Value
  <fct>   <chr> <chr>   <dbl>
1 setosa  Sepal Length    5.1
2 setosa  Sepal Width     3.5
3 setosa  Petal Length    1.4
4 setosa  Petal Width     0.2
5 setosa  Sepal Length    4.9
6 setosa  Sepal Width     3
```

Librería `magrittr`: ceci n'est pas une pipe

Como habrás observado, para conseguir formatear el conjunto `iris` hemos tenido que aplicar dos operaciones, `gather` y `separate`. Esto nos ha obligado a tener que crear una variable, `iris.t`, donde almacenar el resultado de `gather`, para luego pasárselo a `separate`. También podríamos haber hecho algo como:

```
iris.t <- separate(gather(iris, Part, Value, -Species),
                   Part, into=c("Part", "Measure"))
head(iris.t)
```

```
  Species  Part Measure Value
1  setosa Sepal  Length   5.1
2  setosa Sepal  Length   4.9
3  setosa Sepal  Length   4.7
4  setosa Sepal  Length   4.6
5  setosa Sepal  Length   5.0
6  setosa Sepal  Length   5.4
```

Pero queda algo confuso, y aún lo es más cuando en lugar de dos operaciones hemos de realizar varias seguidas.

Una alternativa es usar la librería `magrittr` que define el operador `%>%` que permite encadenar este tipo

acciones. El concepto es que los datos pasan de una función a otra como a través de una tubería (*pipe* en inglés) de funciones. Veamos cómo queda nuestro ejemplo:

```
iris.t <- gather(iris, Part, Value, -Species) %>% separate(Part, into=c("Part", "Measure"))
iris.t %>% head
```

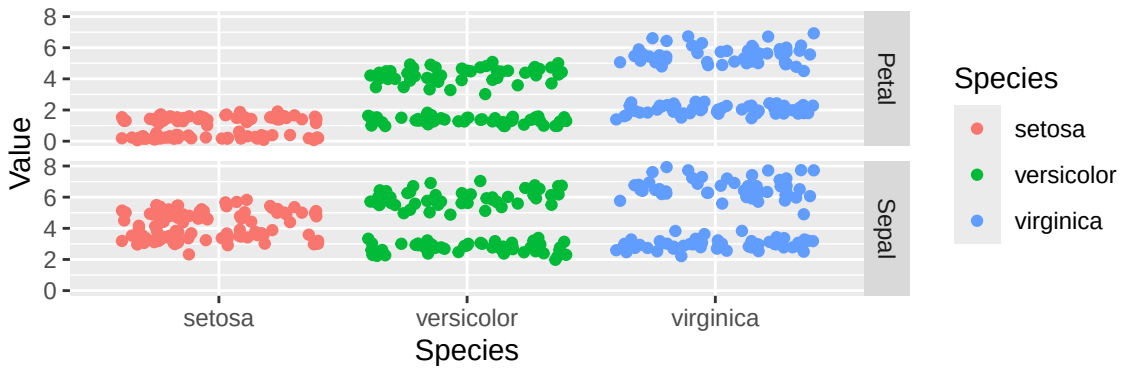
```
  Species  Part Measure Value
1  setosa Sepal  Length   5.1
2  setosa Sepal  Length   4.9
3  setosa Sepal  Length   4.7
4  setosa Sepal  Length   4.6
5  setosa Sepal  Length   5.0
6  setosa Sepal  Length   5.4
```

Fíjate que no hace falta especificar el primer argumento en las funciones `gather`, `separate` y `head`. El operador `%>%` automáticamente lo hace por nosotros.

Ejemplo de visualización

¿Por qué es mejor el formato *tidy* que el original? Una razón es que algunas librerías y funciones esperan encontrar los datos en formato *tidy*, entre ellas `ggplot2`. Por ejemplo, imagina que queremos representar los datos separados en función de la medida realizada, *Sepal o Petal*. Con los datos originales es complicado, puesto que esta información aparece distribuida en cuatro columnas. Con el conjunto *tidy* es inmediato.

```
library(ggplot2)
ggplot(iris.t) + aes(Species, Value, color=Species) +
  geom_jitter() + facet_grid(Part ~ .)
```



En los siguientes ejercicios veremos los ejemplos más típicos de datos desorganizados, o *messy data*:

- Las cabeceras de las columnas son variables en sí mismas, no nombres de variables.
- Más de una variable aparece en una sola columna.
- Las variables aparecen tanto en filas como en columnas.
- Varias unidades de observación están almacenadas en una única tabla.
- Una única unidad de observación está almacenada en varias tablas.

Las cabeceras de las columnas son variables en lugar de nombres de variables

El siguiente conjunto de datos muestra la relación entre los ingresos y la religión en EEUU.

```
print(load('data/pew_religion.Rdata'))
```

```
[1] "pew_religion"      "pew_religion_tidy"
```

```
head(pew_religion[, 1:10])
```

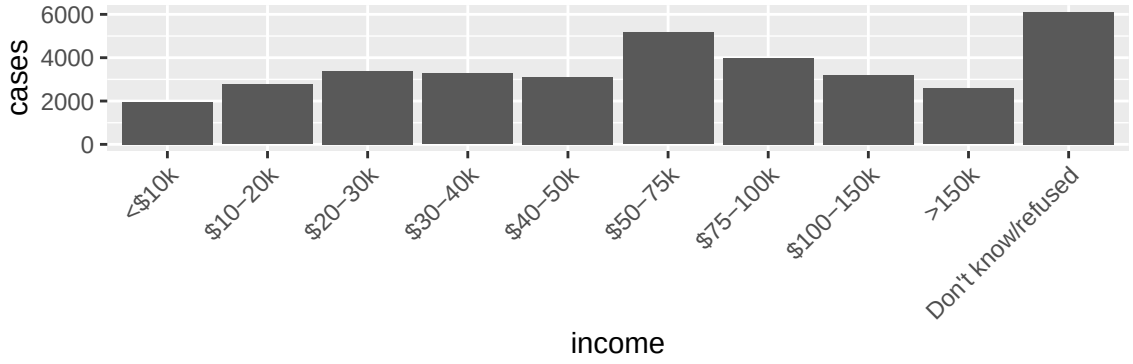
```
            religion <$10k $10-20k $20-30k $30-40k $40-50k $50-75k $75-100k $100-150k >150k
```

```
1           Agnostic    27      34      60      81      76     137     122     109    84
2            Atheist    12      27      37      52      35      70      73      59    74
3           Buddhist    27      21      30      34      33      58      62      39    53
4           Catholic   418     617     732     670     638    1116     949     792   633
5 Don't know/refused    15      14      15      11      10      35      21      17    18
6   Evangelical Prot   575     869    1064     982     881    1486     949     723   414
```

Como se ve, las cabeceras son valores de una variable de tipo factor (los ingresos) en lugar de nombres de variables.

Este conjunto de datos tiene en realidad tres variables, que deberían estar en tres columnas: `religion`, `ingresos` y `frecuencia` (esta última con el número de casos).

Para ver la utilidad de organizar los datos en formato *tidy*, imagina que quisieramos realizar un gráfico donde se mostrará la cantidad de gente por cada intervalo de ingresos:



¿Cómo lo harías con el conjunto original?

Fíjate que este gráfico es mucho más fácil de realizar si en lugar de tener los intervalos de ingresos repartidos en columnas los tuvieramos como una variable.

Para convertir este conjunto a formato *tidy* tendremos que agrupar todas las columnas excepto la de `religion` en dos, una llamada `income` con los intervalos de ingresos, y otra llamada `freq` con el número de casos para cada `income`. Como ya hemos visto en el ejemplo del conjunto IRIS, esto se consigue con `gather`.

Ejercicio 1

Convierte el dataset `pew_religion` a formato *tidy* usando `gather`. La columna `income` debe contener una variable de tipo `factor`.

```
str(pew_religion)
```

```
'data.frame':   18 obs. of  11 variables:
 $ religion          : chr  "Agnostic" "Atheist" "Buddhist" "Catholic" ...
 $ <$10k             : int  27 12 27 418 15 575 1 228 20 19 ...
 $ $10-20k           : int  34 27 21 617 14 869 9 244 27 19 ...
 $ $20-30k           : int  60 37 30 732 15 1064 7 236 24 25 ...
 $ $30-40k           : int  81 52 34 670 11 982 9 238 24 25 ...
 $ $40-50k           : int  76 35 33 638 10 881 11 197 21 30 ...
 $ $50-75k           : int  137 70 58 1116 35 1486 34 223 30 95 ...
 $ $75-100k          : int  122 73 62 949 21 949 47 131 15 69 ...
 $ $100-150k         : int  109 59 39 792 17 723 48 81 11 87 ...
 $ >150k             : int  84 74 53 633 18 414 54 78 6 151 ...
 $ Don't know/refused: int  96 76 54 1489 116 1529 37 339 37 162 ...
```

```
pew_religion_tidy <- gather(pew_religion,
                            key = income,
```

```
                            value = freq,
                            -religion,
                            factor_key = TRUE)

head(pew_religion_tidy)
```

```
            religion income freq
1           Agnostic  <$10k   27
2            Atheist  <$10k   12
3           Buddhist  <$10k   27
4           Catholic  <$10k  418
5 Don't know/refused  <$10k   15
6   Evangelical Prot  <$10k  575
```

```
str(pew_religion_tidy)
```

```
'data.frame':   180 obs. of  3 variables:
 $ religion: chr  "Agnostic" "Atheist" "Buddhist" "Catholic" ...
 $ income  : Factor w/ 10 levels "<$10k","$10-20k",..: 1 1 1 1 1 1 1 1 1 1 ...
 $ freq    : int  27 12 27 418 15 575 1 228 20 19 ...
```

Otro ejemplo usando el conjunto de datos `Billboard` que muestra cuando una canción entró en el top 100.

```
load('data/billboard.Rdata')
head(raw[, 1:12])
```

```
  year       artist                  track time date.entered wk1 wk2 wk3 wk4 wk5 wk6 wk7
1 2000        2 Pac          Baby Don't Cry 4:22   2000-02-26  87  82  72  77  87  94  99
2 2000      2Ge+her The Hardest Part Of ... 3:15   2000-09-02  91  87  92  NA  NA  NA  NA
3 2000 3 Doors Down              Kryptonite 3:53   2000-04-08  81  70  68  67  66  57  54
4 2000 3 Doors Down                   Loser 4:24   2000-10-21  76  76  72  69  67  65  55
5 2000     504 Boyz           Wobble Wobble 3:35   2000-04-15  57  34  25  17  17  31  36
6 2000          98? Give Me Just One Nig... 3:24   2000-08-19  51  39  34  26  26  19   2
```

```
str(raw)
```

```
'data.frame':   317 obs. of  81 variables:
 $ year        : int  2000 2000 2000 2000 2000 2000 2000 2000 2000 2000 ...
 $ artist      : chr  "2 Pac" "2Ge+her" "3 Doors Down" "3 Doors Down" ...
 $ track       : chr  "Baby Don't Cry" "The Hardest Part Of ..." "Kryptonite" "Loser" ...
 $ time        : chr  "4:22" "3:15" "3:53" "4:24" ...
 $ date.entered: chr  "2000-02-26" "2000-09-02" "2000-04-08" "2000-10-21" ...
 $ wk1         : int  87 91 81 76 57 51 97 84 59 76 ...
 $ wk2         : int  82 87 70 76 34 39 97 62 53 76 ...
 $ wk3         : int  72 92 68 72 25 34 96 51 38 74 ...
 $ wk4         : int  77 NA 67 69 17 26 95 41 28 69 ...
 $ wk5         : int  87 NA 66 67 17 26 100 38 21 68 ...
 $ wk6         : int  94 NA 57 65 31 19 NA 35 18 67 ...
 $ wk7         : int  99 NA 54 55 36 2 NA 35 16 61 ...
 $ wk8         : int  NA NA 53 59 49 2 NA 38 14 58 ...
 $ wk9         : int  NA NA 51 62 53 3 NA 38 12 57 ...
 $ wk10        : int  NA NA 51 61 57 6 NA 36 10 59 ...
 $ wk11        : int  NA NA 51 61 64 7 NA 37 9 66 ...
 $ wk12        : int  NA NA 51 59 70 22 NA 37 8 68 ...
 $ wk13        : int  NA NA 47 61 75 29 NA 38 6 61 ...
 $ wk14        : int  NA NA 44 66 76 36 NA 49 1 67 ...
```

```
$ wk15        : int  NA NA 38 72 78 47 NA 61 2 59 ...
$ wk16        : int  NA NA 28 76 85 67 NA 63 2 63 ...
$ wk17        : int  NA NA 22 75 92 66 NA 62 2 67 ...
$ wk18        : int  NA NA 18 67 96 84 NA 67 2 71 ...
$ wk19        : int  NA NA 18 73 NA 93 NA 83 3 79 ...
$ wk20        : int  NA NA 14 70 NA 94 NA 86 4 89 ...
$ wk21        : int  NA NA 12 NA NA NA NA NA 5 NA ...
$ wk22        : int  NA NA 7 NA NA NA NA NA 5 NA ...
$ wk23        : int  NA NA 6 NA NA NA NA NA 6 NA ...
$ wk24        : int  NA NA 6 NA NA NA NA NA 9 NA ...
$ wk25        : int  NA NA 6 NA NA NA NA NA 13 NA ...
$ wk26        : int  NA NA 5 NA NA NA NA NA 14 NA ...
$ wk27        : int  NA NA 5 NA NA NA NA NA 16 NA ...
$ wk28        : int  NA NA 4 NA NA NA NA NA 23 NA ...
$ wk29        : int  NA NA 4 NA NA NA NA NA 22 NA ...
$ wk30        : int  NA NA 4 NA NA NA NA NA 33 NA ...
$ wk31        : int  NA NA 4 NA NA NA NA NA 36 NA ...
$ wk32        : int  NA NA 3 NA NA NA NA NA 43 NA ...
$ wk33        : int  NA NA 3 NA NA NA NA NA NA NA ...
$ wk34        : int  NA NA 3 NA NA NA NA NA NA NA ...
$ wk35        : int  NA NA 4 NA NA NA NA NA NA NA ...
$ wk36        : int  NA NA 5 NA NA NA NA NA NA NA ...
$ wk37        : int  NA NA 5 NA NA NA NA NA NA NA ...
$ wk38        : int  NA NA 9 NA NA NA NA NA NA NA ...
$ wk39        : int  NA NA 9 NA NA NA NA NA NA NA ...
$ wk40        : int  NA NA 15 NA NA NA NA NA NA NA ...
$ wk41        : int  NA NA 14 NA NA NA NA NA NA NA ...
$ wk42        : int  NA NA 13 NA NA NA NA NA NA NA ...
$ wk43        : int  NA NA 14 NA NA NA NA NA NA NA ...
$ wk44        : int  NA NA 16 NA NA NA NA NA NA NA ...
$ wk45        : int  NA NA 17 NA NA NA NA NA NA NA ...
$ wk46        : int  NA NA 21 NA NA NA NA NA NA NA ...
$ wk47        : int  NA NA 22 NA NA NA NA NA NA NA ...
$ wk48        : int  NA NA 24 NA NA NA NA NA NA NA ...
$ wk49        : int  NA NA 28 NA NA NA NA NA NA NA ...
$ wk50        : int  NA NA 33 NA NA NA NA NA NA NA ...
$ wk51        : int  NA NA 42 NA NA NA NA NA NA NA ...
$ wk52        : int  NA NA 42 NA NA NA NA NA NA NA ...
$ wk53        : int  NA NA 49 NA NA NA NA NA NA NA ...
$ wk54        : int  NA NA NA NA NA NA NA NA NA NA ...
$ wk55        : int  NA NA NA NA NA NA NA NA NA NA ...
$ wk56        : int  NA NA NA NA NA NA NA NA NA NA ...
$ wk57        : int  NA NA NA NA NA NA NA NA NA NA ...
$ wk58        : int  NA NA NA NA NA NA NA NA NA NA ...
$ wk59        : int  NA NA NA NA NA NA NA NA NA NA ...
$ wk60        : int  NA NA NA NA NA NA NA NA NA NA ...
$ wk61        : int  NA NA NA NA NA NA NA NA NA NA ...
$ wk62        : int  NA NA NA NA NA NA NA NA NA NA ...
$ wk63        : int  NA NA NA NA NA NA NA NA NA NA ...
$ wk64        : int  NA NA NA NA NA NA NA NA NA NA ...
$ wk65        : int  NA NA NA NA NA NA NA NA NA NA ...
$ wk66        : logi  NA NA NA NA NA NA ...
$ wk67        : logi  NA NA NA NA NA NA ...
$ wk68        : logi  NA NA NA NA NA NA ...
```

```
 $ wk69        : logi  NA NA NA NA NA NA ...
 $ wk70        : logi  NA NA NA NA NA NA ...
 $ wk71        : logi  NA NA NA NA NA NA ...
 $ wk72        : logi  NA NA NA NA NA NA ...
 $ wk73        : logi  NA NA NA NA NA NA ...
 $ wk74        : logi  NA NA NA NA NA NA ...
 $ wk75        : logi  NA NA NA NA NA NA ...
 $ wk76        : logi  NA NA NA NA NA NA ...
```

La posición (*ranking*) de cada canción en cada semana después de haber entrado en el Top 100 se almacena en las columnas `wk1` a `wk76` (ver `str(raw)`). Si un tema sale del Top 100 antes de 76 semanas se rellenan los datos con NA's.

Para obtener la versión *tidy* hemos de juntar todas las columnas desde `wk1` a `wk76` asignando como clave (`key`) una variable de nombre `week`, como valor (`value`) otra variable de nombre `rank`.

```
billboard_tidy <- gather(raw,
                         key = week,
                         value = rank,
                         wk1:wk76)

head(billboard_tidy)
```

```
  year       artist                    track time date.entered week rank
1 2000        2 Pac           Baby Don't Cry 4:22   2000-02-26  wk1   87
2 2000      2Ge+her The Hardest Part Of ... 3:15   2000-09-02  wk1   91
3 2000 3 Doors Down               Kryptonite 3:53   2000-04-08  wk1   81
4 2000 3 Doors Down                    Loser 4:24   2000-10-21  wk1   76
5 2000     504 Boyz            Wobble Wobble 3:35   2000-04-15  wk1   57
6 2000          98? Give Me Just One Nig... 3:24   2000-08-19  wk1   51
```

```
sum(is.na(billboard_tidy))
```

```
[1] 18785
```

Observamos que los valores `NA` se mantienen, ya que indican las semanas en las que una canción no entra en el Top 100 y, además, no se ha pedido eliminarlos.

Ejercicio 2

Obtén la versión *tidy* de este conjunto de datos usando `pivot_longer`.

Consejo: puedes eliminar el prefijo `wk` de las semanas mediante el argumento `names_prefix`. Opcionalmente puedes crear una columna `date` a partir de `date.entered`, sumando las semanas transcurridas desde la fecha de entrada en lista.

```
billboard_tidy2 <- raw %>%
  pivot_longer(
    cols = wk1:wk76,
    names_to = "week",
    values_to = "rank",
    names_prefix = "wk"
  ) %>%
  mutate(week = as.integer(week))

head(billboard_tidy2)
```

```
# A tibble: 6 x 7
```

```
   year artist track          time  date.entered  week  rank
  <int> <chr>  <chr>          <chr> <chr>        <int> <int>
1  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       1    87
2  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       2    82
3  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       3    72
4  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       4    77
5  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       5    87
6  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       6    94
```

El resultado es equivalente al obtenido con `gather()`. No obstante, como se ha eliminado el prefijo y cada semana se ve representada por un entero, se ha castedo la columna `week` a **integer**.

```
billboard_tidy2 <- billboard_tidy2 %>%
  mutate(
    week = as.integer(week),
    date = as.Date(date.entered) + 7 * (week - 1)
  )

billboard_tidy2
```

```
# A tibble: 24,092 x 8
    year artist track          time  date.entered  week  rank date
   <int> <chr>  <chr>          <chr> <chr>        <int> <int> <date>
 1  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       1    87 2000-02-26
 2  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       2    82 2000-03-04
 3  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       3    72 2000-03-11
 4  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       4    77 2000-03-18
 5  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       5    87 2000-03-25
 6  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       6    94 2000-04-01
 7  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       7    99 2000-04-08
 8  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       8    NA 2000-04-15
 9  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       9    NA 2000-04-22
10  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26      10    NA 2000-04-29
# i 24,082 more rows
```

Varias variables en una misma columna

El conjunto de datos `tuberculosis`, de la *World Health Organization*, presenta este tipo de desordenación (además de la vista en el apartado anterior). Muestra el número de casos de tuberculosis por país, año y grupo demográfico. Estos grupos demográficos se muestran en los nombres de las columnas como `m` o `f` (sexo) y la edad (0-14, 15-24, 25-34, 35-44, 45-54, 55-64, 65- y desconocido (`u`)).

```
load('data/tb.Rdata')
head(raw[, 1:16])
```

```
    country year m014 m1524 m2534 m3544 m4554 m5564 m65 mu f014 f1524 f2534 f3544 f4554 f5564
11       AD 2000    0     0     1     0     0     0   0 NA   NA    NA    NA    NA    NA    NA
37       AE 2000    2     4     4     6     5    12  10 NA    3    16     1     3     0     0
61       AF 2000   52   228   183   149   129    94  80 NA   93   414   565   339   205    99
88       AG 2000    0     0     0     0     0     0   1 NA    1     1     1     0     0     0
137      AL 2000    2    19    21    14    24    19  16 NA    3    11    10     8     8     5
166      AM 2000    2   152   130   131    63    26  21 NA    1    24    27    24     8     8
```

Podemos ver como efectivamente en una columna tenemos dos variables juntas, el sexo y el rango de edad. Por ejemplo, en `m014` la `m` hace referencia al sexo y `014` al rango de edad, dos variables que deberían estar separadas.

Para pasar a formato *tidy* necesitamos realizar dos operaciones en este caso. En la primera juntaremos todas las columnas `m...` y `f...` y los conteos (los llamaremos `casos`). En la segunda, separaremos las columnas con los datos de tipo `m014` en dos columnas, `sexo` y `edad`.

Ejercicio 3

Convierte el conjunto de datos TB en *tidy* utilizando `gather` y `separate`.

```
str(raw)
```

```
'data.frame':   201 obs. of  18 variables:
 $ country: chr  "AD" "AE" "AF" "AG" ...
 $ year   : int  2000 2000 2000 2000 2000 2000 2000 2000 2000 2000 ...
 $ m014   : int  0 2 52 0 2 2 0 186 97 NA ...
 $ m1524  : int  0 4 228 0 19 152 0 999 278 NA ...
 $ m2534  : int  1 4 183 0 21 130 1 1003 594 NA ...
 $ m3544  : int  0 6 149 0 14 131 2 912 402 NA ...
 $ m4554  : int  0 5 129 0 24 63 0 482 419 1 ...
 $ m5564  : int  0 12 94 0 19 26 0 312 368 1 ...
 $ m65    : int  0 10 80 1 16 21 0 194 330 NA ...
 $ mu     : int  NA NA NA NA NA NA NA NA NA NA ...
 $ f014   : int  NA 3 93 1 3 1 0 247 121 NA ...
 $ f1524  : int  NA 16 414 1 11 24 0 1142 544 NA ...
 $ f2534  : int  NA 1 565 1 10 27 1 1091 479 NA ...
 $ f3544  : int  NA 3 339 0 8 24 0 844 262 NA ...
 $ f4554  : int  NA 0 205 0 8 8 0 417 230 1 ...
 $ f5564  : int  NA 0 99 0 5 8 1 200 179 NA ...
 $ f65    : int  NA 4 36 0 11 4 0 120 216 NA ...
 $ fu     : int  NA NA NA NA NA NA NA NA NA NA ...
```

```
tb_tidy <- raw %>%
  gather(key = grupo, value = casos, -country, -year) %>%
  separate(col = grupo, into = c("sexo", "edad"), sep = 1)

head(raw)
```

```
    country year m014 m1524 m2534 m3544 m4554 m5564 m65 mu f014 f1524 f2534 f3544 f4554 f5564 f65
11       AD 2000    0     0     1     0     0     0   0 NA   NA    NA    NA    NA    NA    NA  NA
37       AE 2000    2     4     4     6     5    12  10 NA    3    16     1     3     0     0   4
61       AF 2000   52   228   183   149   129    94  80 NA   93   414   565   339   205    99  36
88       AG 2000    0     0     0     0     0     0   1 NA    1     1     1     0     0     0   0
137      AL 2000    2    19    21    14    24    19  16 NA    3    11    10     8     8     5  11
166      AM 2000    2   152   130   131    63    26  21 NA    1    24    27    24     8     8   4
    fu
11  NA
37  NA
61  NA
88  NA
137 NA
166 NA
```

```
head(tb_tidy)
```

```
  country year sexo edad casos
1      AD 2000    m  014     0
2      AE 2000    m  014     2
3      AF 2000    m  014    52
```

```
4      AG 2000    m  014     0
5      AL 2000    m  014     2
6      AM 2000    m  014     2
```

Obtenemos el conjunto en formato `tidy`, donde cada fila representa el número de casos de tuberculosis para un país, año, sexo y rango de edad determinados.

Variables almacenadas tanto en filas como en columnas

Esta es la forma más complicada de datos desordenados, y es lo que ocurre en el conjunto de datos del *Global Historical Climatology Network*. Se muestra el tiempo para la estación `MX17004`, en México, durante 5 meses del año 2010.

```
load('data/weather.Rdata')
head(raw[, 1:20])
```

```
       id year month element d1   d2   d3 d4   d5 d6 d7 d8 d9  d10  d11 d12 d13 d14 d15  d16
1 MX17004 2010     1    tmax NA   NA   NA NA   NA NA NA NA NA   NA   NA  NA  NA  NA  NA   NA
2 MX17004 2010     1    tmin NA   NA   NA NA   NA NA NA NA NA   NA   NA  NA  NA  NA  NA   NA
3 MX17004 2010     2    tmax NA 27.3 24.1 NA   NA NA NA NA NA   NA 29.7  NA  NA  NA  NA   NA
4 MX17004 2010     2    tmin NA 14.4 14.4 NA   NA NA NA NA NA   NA 13.4  NA  NA  NA  NA   NA
5 MX17004 2010     3    tmax NA   NA   NA NA 32.1 NA NA NA NA 34.5   NA  NA  NA  NA  NA 31.1
6 MX17004 2010     3    tmin NA   NA   NA NA 14.2 NA NA NA NA 16.8   NA  NA  NA  NA  NA 17.6
```

Como se observa, además de las variables `id`, `year` y `month`, tiene variables en las columnas `d1` a `d31` que deberían corresponder a una única variable `dia`, y en las filas, `tmin` y `tmax` deberían estar como columnas. Fíjate además que la columna `element` no es una variable, sino que almacena nombres de variables.

Para obtener la versión *tidy* primero juntamos todos los días en una variable `dia`. Luego tendremos que separar la columna `element` en dos columnas, `tmax` y `tmin`. Esto último lo lograremos con la función `pivot_wider` de tidyr, que no hemos utilizado hasta ahora.

Ejercicio 4

Obtén la versión *tidy* del conjunto de datos `weather` utilizando las funciones `pivot_longer` y `pivot_wider`. Además cambiaremos las columnas `year`, `month` y `dia` por una única variable `fecha`. Para que la operación `pivot_wider` no de error tendremos que eliminar los NA's cuando usemos `pivot_longer`.

Al igual que en el ejercicio 2, aunque no lo pide, añadimos el parámetro `names_prefix()` para eliminar el prefijo `d` asociado a cada uno de los días, y además, convertimos la nueva columna a `integer` ya que, por defecto, es `character`.

```
weather_tidy <- raw %>%
  pivot_longer(   # las columnas d1 a d31 agrupadas en una única variable dia
    cols = d1:d31,
    names_to = "dia",
    values_to = "temperatura",
    names_prefix = "d",
    values_drop_na = TRUE
  ) %>% # variable dia a tipo entero
  mutate(
    dia = as.integer(dia)
  ) %>% # las temperaturas máximas y mínimas separadas en dos columnas
  pivot_wider(
    names_from = element,
    values_from = temperatura
  )
```

```r
head(weather_tidy)
```

```
# A tibble: 6 x 6
  id       year month   dia  tmax  tmin
  <chr>   <int> <int> <int> <dbl> <dbl>
1 MX17004  2010     1    30  27.8  14.5
2 MX17004  2010     2     2  27.3  14.4
3 MX17004  2010     2     3  24.1  14.4
4 MX17004  2010     2    11  29.7  13.4
5 MX17004  2010     2    23  29.9  10.7
6 MX17004  2010     3     5  32.1  14.2
```

```r
# year, month y dia en una única variable fecha
weather_tidy <- weather_tidy %>%
  unite("fecha", year, month, dia, sep = "-") %>%
  mutate(fecha = as.Date(fecha))

head(weather_tidy)
```

```
# A tibble: 6 x 4
  id      fecha       tmax  tmin
  <chr>   <date>     <dbl> <dbl>
1 MX17004 2010-01-30  27.8  14.5
2 MX17004 2010-02-02  27.3  14.4
3 MX17004 2010-02-03  24.1  14.4
4 MX17004 2010-02-11  29.7  13.4
5 MX17004 2010-02-23  29.9  10.7
6 MX17004 2010-03-05  32.1  14.2
```

DPLYR

Introducción y objetivos.

El objetivo de esta práctica es manejar data frames estructurados (variables en columnas, muestras en filas) con **tidyR** de una manera eficiente y sencilla utilizando la librería **DPLYR**. Esta librería fue desarrollada por Hadley Wickham (el creador de tidyR) con funciones programadas en C++ de manera óptima. Cuando hablamos de manejar data frames nos referimos a realizar operaciones sobre las muestras y sobre las variables tales como:

1. Seleccionar muestras de un data frame en base a algún tipo de criterio.
2. Seleccionar variables que cumplan alguna condición.
3. Ordenar las muestras de un data frame.
4. Unir varios data frames.
5. Crear nuevas variables a partir de las variables existentes.
6. Agrupar muestras que cumplan algún criterio para realizar cálculos estadísticos.

Para llevar a cabo estas operaciones **DPLYR** proporciona una serie de funciones que enumeramos a continuación:

- `select()`. Permite seleccionar un subconjunto de las variables (columnas) del data frame.
- `filter()`. Esta función, como su propio nombre indica, permite filtrar las muestras (filas) de un data frame en base a algún criterio.
- `group_by()`. Esta función crea grupos de muestras a partir de las categorías de una variable.
- `summarise()`. Esta función facilita el cálculo de parámetros estadísticos de las variables de un data frame.

- `arrange()`. Permite reordenar las muestras de un data frame.
- `XXX_join()`. Las funciones de la familia *XXX_join* permiten combinar data frames.
- `rename()`. Esta función permite renombrar variables de un data frame.
- `mutate()`. Esta función permite añadir nuevas variables (columnas) a partir de la transformación de columnas ya existentes.
- `%>%` Es el operador *pipe* que permite concatenar funciones.

En clase de teoría te han explicado cómo utilizar estas funciones y ya has trabajado con ellas, así que vamos a ver como te desenvuelves con **DPLYR**.

Vamos a cargar el conjunto de datos `VuelosNY.Rdata` que contiene dos data frames: `flights` y `planes`.

Ejercicio 5

Muestra las primeras muestras de ambos data frames para hacerte una idea de las variables que contienen. ¿Qué diferencia existe entre la información mostrada en ambos data frames?

Cargamos el archivo .Rdata sobre una variable para visualizar que lo compone.

```
data <- load("./data/VuelosNY.Rdata")
data
```

```
[1] "planes"  "flights"
```

Observamos que está compuesto por dos data frames distintos.

```
head(flights)
```

```
  year month day dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_delay carrier flight
1 2013     1   1      517            515         2      830            819        11      UA   1545
2 2013     1   1      533            529         4      850            830        20      UA   1714
3 2013     1   1      542            540         2      923            850        33      AA   1141
4 2013     1   1      544            545        -1     1004           1022       -18      B6    725
5 2013     1   1      554            600        -6      812            837       -25      DL    461
6 2013     1   1      554            558        -4      740            728        12      UA   1696
  tailnum origin dest air_time distance hour minute  time_hour
1  N14228    EWR  IAH      227     1400    5     15 1357034400
2  N24211    LGA  IAH      227     1416    5     29 1357034400
3  N619AA    JFK  MIA      160     1089    5     40 1357034400
4  N804JB    JFK  BQN      183     1576    5     45 1357034400
5  N668DN    LGA  ATL      116      762    6      0 1357038000
6  N39463    EWR  ORD      150      719    5     58 1357034400
```

```
head(planes)
```

```
  tailnum year                type     manufacturer     model engines seats speed    engine
1  N10156 2004 Fixed wing multi engine          EMBRAER EMB-145XR       2    55    NA Turbo-fan
2  N102UW 1998 Fixed wing multi engine AIRBUS INDUSTRIE  A320-214       2   182    NA Turbo-fan
3  N103US 1999 Fixed wing multi engine AIRBUS INDUSTRIE  A320-214       2   182    NA Turbo-fan
4  N104UW 1999 Fixed wing multi engine AIRBUS INDUSTRIE  A320-214       2   182    NA Turbo-fan
5  N10575 2002 Fixed wing multi engine          EMBRAER EMB-145LR       2    55    NA Turbo-fan
6  N105UW 1999 Fixed wing multi engine AIRBUS INDUSTRIE  A320-214       2   182    NA Turbo-fan
```

```
dim(flights)
```

```
[1] 336776     19
```

```
dim(planes)
```

```
[1] 3322    9
```

Tras visualizar el contenido de ambos, vemos que `flights` contiene **19 variables** relacionadas con **vuelos**, como la fecha o las horas de salida y llegada.

Por otro lado, `planes` contiene **9 variables** que describen las **características de los aviones**. Por tanto, la principal diferencia es que cada dataset no representa lo mismo. No obstante, si que pueden relacionarse por medio de la variable `tailnum`, que refiere a la matrícula del avión, presente tanto en `planes` como en `flights`.

Ejercicio 6

En nuestro estudio queremos aplicar una serie de criterios de exclusión sobre las muestras de ambos data frames. En concreto queremos eliminar los aviones fabricados por el fabricante MCDONELL DOUGLAS (vigila si existe más de una denominación de esta compañía en la variable `manufacturer`). Del mismo modo, queremos seleccionar aquellos aviones en los que el número de asientos esté entre 90 y 210 (variable `seats`). Aplica las dos operaciones de filtrado secuencialmente utilizando el operador *pipe* y almacena el data frame resultante en `planes.f`. En el data frame resultante unifica las denominaciones del fabricante `AIRBUS` bajo la denominación `"AIRBUS COMP"`.

- Nota: Revisar el uso de la función filter vista en clase y tener en cuenta los operadores de comparación: igual ==, distinto !=, mayor >, mayor o igual >=, menor <, menor o igual <=.

Como se nos pide excluir los aviones fabricados por una compañía concreta, utilizamos `unique()` para obtener los distintos fabricantes presentes en el conjunto de datos. Seguidamente, con `grep()` identificamos todas las denominaciones que contienen **MCDONNELL** y comprobar si la compañía aparece registrada bajo diferentes nombres.

De la misma manera, buscamos las denominaciones que contienen **AIRBUS**, ya que posteriormente vamos a unificarlas bajo un mismo nombre.

```
mcdonnell <- grep("MCDONNELL", unique(planes$manufacturer), value = TRUE)
airbus <- grep("AIRBUS", unique(planes$manufacturer), value = TRUE)

mcdonnell
```

```
[1] "MCDONNELL DOUGLAS"             "MCDONNELL DOUGLAS CORPORATION" "MCDONNELL DOUGLAS AIRCRAFT CO"
```

```
airbus
```

```
[1] "AIRBUS INDUSTRIE" "AIRBUS"
```

```
# excluimos el fabricante indicado y seleccionamos aviones de 90 a 210 asientos
planes.f <- planes %>%
  filter(!manufacturer %in% mcdonnell) %>%
  filter(seats >= 90, seats <= 210) %>% # unificamos las denominaciones de AIRBUS en AIRBUS COMP
  mutate(manufacturer = ifelse(manufacturer %in% airbus, "AIRBUS COMP", manufacturer))

head(planes.f)
```

```
  tailnum year                type manufacturer    model engines seats speed    engine
1  N102UW 1998 Fixed wing multi engine  AIRBUS COMP A320-214       2   182    NA Turbo-fan
2  N103US 1999 Fixed wing multi engine  AIRBUS COMP A320-214       2   182    NA Turbo-fan
3  N104UW 1999 Fixed wing multi engine  AIRBUS COMP A320-214       2   182    NA Turbo-fan
4  N105UW 1999 Fixed wing multi engine  AIRBUS COMP A320-214       2   182    NA Turbo-fan
5  N107US 1999 Fixed wing multi engine  AIRBUS COMP A320-214       2   182    NA Turbo-fan
6  N108UW 1999 Fixed wing multi engine  AIRBUS COMP A320-214       2   182    NA Turbo-fan
```

Ejercicio 7

Crea un data frame denominado `flights.f` que contenga los vuelos que involucren a los aviones presentes en el data frame `planes.f`. Para ello utiliza algunas de las funciones de **XXX_join** y la variable `tailnum` que es común en ambos data frames.

```
flights.f <- flights %>%
  semi_join(planes.f, by = "tailnum")

head(flights.f)
```

```
  year month day dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_delay carrier flight
1 2013     1   1      517            515         2      830            819        11      UA   1545
2 2013     1   1      533            529         4      850            830        20      UA   1714
3 2013     1   1      542            540         2      923            850        33      AA   1141
4 2013     1   1      544            545        -1     1004           1022       -18      B6    725
5 2013     1   1      554            600        -6      812            837       -25      DL    461
6 2013     1   1      554            558        -4      740            728        12      UA   1696
  tailnum origin dest air_time distance hour minute  time_hour
1  N14228    EWR  IAH      227     1400    5     15 1357034400
2  N24211    LGA  IAH      227     1416    5     29 1357034400
3  N619AA    JFK  MIA      160     1089    5     40 1357034400
4  N804JB    JFK  BQN      183     1576    5     45 1357034400
5  N668DN    LGA  ATL      116      762    6      0 1357038000
6  N39463    EWR  ORD      150      719    5     58 1357034400
```

La función `semi_join()` incluye filas de `x` que coinciden con `y` pero solo mantiene las de `x`. Como solamente queremos seleccionar vuelos, no añadir columnas de **planes.f**, nos permite seleccionar unicamente aquellos vuelos cuyos aviones se encuentren en **planes.f**, relacionando ambos datasets mediante la variable `tailnum`, que ya vimos anteriormente que es común entre ellos.

Como resultado, obtenemos un nuevo dataset **flights.f**.

Ejercicio 8

Reasigna los nombres de los data frames `planes.f` y `flights.f` a los nombres originales `planes` y `flights`.

Resolvemos el ejercicio volcando directamente el contenido de `planes.f` y `flights.f` sobre `planes` y `flights`.

```
planes <- planes.f
flights <- flights.f

all.equal(planes, planes.f)
```

```
[1] TRUE
```

```
all.equal(flights, flights.f)
```

```
[1] TRUE
```

Comprobamos si ambos tienen el mismo contenido y observamos que **sí lo comparten**.

Ejercicio 9

Existen variables que, a priori, no tienen demasiado interés para el estudio que queremos llevar a cabo, en particular queremos eliminar:

- En el data frame `flights` quédate con todas las variables excepto `time_hour` y las variables que terminan con la palabra "time".

Utilizamos la función `select()` de `dplyr` para eliminar las variables que no nos interesan. Para ello, añadimos un signo negativo `-` como prefijo de la variable que queremos eliminar.

Por otro lado, como queremos eliminar todo aquello que acabe en **"time"**, hacemos uso de la función `ends_with()`, que como indica, selecciona toda aquella cadena que acabe en el string indicado.

```
flights <- flights %>%
  select(-time_hour, -ends_with("time"))
```

- En el data frame `planes` quédate con todas las variables excepto `speed` y `type`.

```
planes <- planes %>%
  select(-speed, -type)
```

Finalmente, comprobamos mediante `names()` que las variables indicadas se han eliminado correctamente.

```
names(flights)
```

```
 [1] "year"      "month"     "day"       "dep_delay" "arr_delay" "carrier"   "flight"    "tailnum"
 [9] "origin"    "dest"      "distance"  "hour"      "minute"
```

```
names(planes)
```

```
[1] "tailnum"      "year"         "manufacturer" "model"        "engines"      "seats"
[7] "engine"
```

Ejercicio 10

Incorpora una nueva variable a `flights` denominado `Full_Date` que sea de tipo `character` que incorpore una cadena de texto de la forma `d-m-Y`. Donde `d` indica día, `m` mes e `Y` año, a partir de la información incluida en el data frame. Utiliza una función de `DPLYR` que permita conseguir este propósito. Elimina las variables originales involucradas en la creación de la nueva variable

Para crear la variable `Full_Date`, la función propia de la libreria `dplyr` que permite añadir una nueva columna al dataframe es `mutate()`. Como esta debe respetar el formato indicado en el enunciado, con `paste()` concatenamos los valores de las variables `day`, `month` y `year`, separados mediante `-`, generando una cadena de texto.

```
flights <- flights %>%
  mutate(Full_Date = paste(day, month, year, sep = "-")) %>% # creamos la col
  select(-day, -month, -year) # eliminamos las var utilizadas para Full_Date

str(flights)
```

```
'data.frame':   171879 obs. of  11 variables:
 $ dep_delay: num  2 4 2 -1 -6 -4 -5 -3 -2 -2 ...
 $ arr_delay: num  11 20 33 -18 -25 12 19 -8 -2 -3 ...
 $ carrier  : chr  "UA" "UA" "AA" "B6" ...
 $ flight   : int  1545 1714 1141 725 461 1696 507 79 49 71 ...
 $ tailnum  : chr  "N14228" "N24211" "N619AA" "N804JB" ...
 $ origin   : chr  "EWR" "LGA" "JFK" "JFK" ...
 $ dest     : chr  "IAH" "IAH" "MIA" "BQN" ...
 $ distance : num  1400 1416 1089 1576 762 ...
 $ hour     : num  5 5 5 5 6 5 6 6 6 6 ...
 $ minute   : num  15 29 40 45 0 58 0 0 0 0 ...
 $ Full_Date: chr  "1-1-2013" "1-1-2013" "1-1-2013" "1-1-2013" ...
```

Ejercicio 11

Utilizando las funciones `group_by`, `summarise` y `filter`, crea un data frame donde se calcule el número de vuelos (`Num_flights`), la distancia media recorrida (`Mean_distance`) y la distancia total recorrida (`Total_distance`) para cada uno de los aviones (`tailnum`) presentes en el conjunto `flights`. Descarta aquellos registros del data frame en los que la variable `tailnum` tenga un valor NA. Utiliza el operador pipe para enlazar las funciones. Almacena el data frame en una variable denominada `Distances_plane`.

Primero, descartamos los registros cuya matricula sea `NA` mediante `filter()` y agrupamos los vuelos por avión con `group_by()`. A partir de aqui, se calculan las medidas estadísticas solicitadas con `summarise()`, añadiendo `na.rm = TRUE` para evitar obtener resultados nulos ya que, cualquier operación con un NA es NA.

```
Distances_plane <- flights %>%
  filter(!is.na(tailnum)) %>% # nos quedamos con los registros en los que la matricula no es NA
  group_by(tailnum) %>%
  summarise(
    Num_flights = n(), # devuelve el tamaño del group by
    Mean_distance = mean(distance, na.rm = TRUE),
    Total_distance = sum(distance, na.rm = TRUE)
  )

head(Distances_plane)
```

```
# A tibble: 6 x 4
  tailnum Num_flights Mean_distance Total_distance
  <chr>         <int>         <dbl>          <dbl>
1 N102UW           48          536.          25722
2 N103US           46          535.          24619
3 N104UW           47          535.          25157
4 N105UW           45          525.          23618
5 N107US           41          529.          21677
6 N108UW           60          534.          32070
```

Ejercicios Extra

TidyR. Varias unidades de observación en una sola tabla

Volvemos al ejemplo del conjunto `Billboard` para ilustrar este caso. Si nos fijamos en la versión *tidy* que hemos obtenido en el ejercicio 2 vemos como `artist` y `time` se repiten para cada canción (`track`) y semana. Para evitar estas repeticiones hemos de dividir el dataframe en dos, uno de canciones donde almacenaremos el artista, el nombre de la canción y el tiempo, y otro con el *ranking* de canciones que nos dará la posición en el Top 100, la canción y la semana. A esta división de los datos en varias tablas para evitar duplicados se le llama *normalización*, y es habitual cuando estamos trabajando con datos almacenados en bases de datos, si bien no lo es tanto en el campo del análisis de datos, donde muchas veces para analizarlos se requiere volver a juntar de nuevo los datos en una única tabla.

Ejercicio Extra 1

Separa los datos *tidy* que teníamos en el ejercicio 2 en dos tablas (data frames) tal y como se describe en el párrafo anterior y empleando la función `unique` para eliminar repeticiones.

Opcionalmente se puede crear un identificador, `id`, asociado al nombre de la canción (`track`), y en la tabla de ranking solo usaremos este identificador en lugar del nombre de la canción.

```
head(billboard_tidy2)
```

```
# A tibble: 6 x 8
   year artist track          time  date.entered  week  rank date
  <int> <chr>  <chr>          <chr> <chr>        <int> <int> <date>
1  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       1    87 2000-02-26
2  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       2    82 2000-03-04
3  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       3    72 2000-03-11
4  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       4    77 2000-03-18
5  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       5    87 2000-03-25
6  2000 2 Pac  Baby Don't Cry 4:22  2000-02-26       6    94 2000-04-01
```

En primer lugar, creamos el primer dataset `canciones` seleccionando las variables indicadas en el enunciado con `select()` y obviando el resto. Seguidamente, sobre el resultado aplicamos `unique()` para eliminar todos los duplicados y, finalmente, creamos una nueva columna `id` cuyos valores se corresponden con un identificador asociado a cada canción, obtenido mediante `row_number()`.

```
# seleccionamos los datos de cada canción y eliminamos duplicados
canciones <- billboard_tidy2 %>%
  select(artist, track, time) %>%
  unique() %>%
  mutate(id = row_number()
)
```

Para la segunda tabla, volcamos el contenido de `billboard_tidy` y, mediante `left_join()`, incorporamos el identificador `id` de la tabla `canciones`, relacionando ambas tablas a través de las variables comunes (`artist`, `track` y `time`).

Finalmente, con `select()` nos quedamos únicamente `id`, `week` y `rank`, evitando repetir la información de las canciones en cada observación del ranking.

```
ranking <- billboard_tidy2 %>%
  left_join(canciones, by = c("artist", "track", "time")) %>%
  select(id, week, rank)
```

```
head(canciones)
```

```
# A tibble: 6 x 4
  artist       track                   time     id
  <chr>        <chr>                   <chr> <int>
1 2 Pac        Baby Don't Cry          4:22      1
2 2Ge+her      The Hardest Part Of ... 3:15      2
3 3 Doors Down Kryptonite              3:53      3
4 3 Doors Down Loser                   4:24      4
5 504 Boyz     Wobble Wobble           3:35      5
6 98?          Give Me Just One Nig... 3:24      6
```

```
head(ranking)
```

```
# A tibble: 6 x 3
     id  week  rank
  <int> <int> <int>
1     1     1    87
2     1     2    82
3     1     3    72
4     1     4    77
5     1     5    87
6     1     6    94
```

TidyR. Una única unidad de observación almacenada en varias tablas

También nos podemos encontrar casos donde las observaciones están distribuidas en varias tablas. El procedimiento general para abordar estos casos es:

1. Leer todos los ficheros en una lista de tablas.
2. Añadir una nueva columna a cada tabla con el nombre del fichero original.
3. Combinar todas las tablas en una.

Esto se puede hacer fácilmente con las librerías `plyr` y `dplyr`.

TidyR. Otro conjunto messy...

Ejercicio Extra 2

En la librería *tidyr* hay varios conjuntos de datos. **table1** es un conjunto *tidy* pero **table5** no lo es. Realiza las operaciones necesarias para obtener **table1** a partir de **table5**. También deben coincidir los tipos de datos de las columnas.

Antes de procesar `table5`, visualizamos el contenido de ambos datasets para detectar las posibles diferencias y/o coincidencias entre ellos.

Para ello, usamos `head()` y `str()`.

```
head(table1)
```

```
# A tibble: 6 x 4
  country      year  cases population
  <chr>       <dbl>  <dbl>      <dbl>
1 Afghanistan  1999    745   19987071
2 Afghanistan  2000   2666   20595360
3 Brazil       1999  37737  172006362
4 Brazil       2000  80488  174504898
5 China        1999 212258 1272915272
6 China        2000 213766 1280428583
```

```
head(table5)
```

```
# A tibble: 6 x 4
  country     century year  rate
  <chr>       <chr>   <chr> <chr>
1 Afghanistan 19      99    745/19987071
2 Afghanistan 20      00    2666/20595360
3 Brazil      19      99    37737/172006362
4 Brazil      20      00    80488/174504898
5 China       19      99    212258/1272915272
6 China       20      00    213766/1280428583
```

```
str(table1)
```

```
tibble [6 x 4] (S3: tbl_df/tbl/data.frame)
 $ country   : chr [1:6] "Afghanistan" "Afghanistan" "Brazil" "Brazil" ...
 $ year      : num [1:6] 1999 2000 1999 2000 1999 ...
 $ cases     : num [1:6] 745 2666 37737 80488 212258 ...
 $ population: num [1:6] 2.00e+07 2.06e+07 1.72e+08 1.75e+08 1.27e+09 ...
```

```
str(table5)
```

```
tibble [6 x 4] (S3: tbl_df/tbl/data.frame)
 $ country: chr [1:6] "Afghanistan" "Afghanistan" "Brazil" "Brazil" ...
```

```
 $ century: chr [1:6] "19" "20" "19" "20" ...
 $ year   : chr [1:6] "99" "00" "99" "00" ...
 $ rate   : chr [1:6] "745/19987071" "2666/20595360" "37737/172006362" "80488/174504898" ...
```

Observamos que, tal y como indica el enunciado, `table1` tiene una estructura *tidy*, con las variables `country`, `year`, `cases` y `population` en columnas independientes.

En cambio, en `table5` el año aparece dividido entre `century` y `year`, mientras que `rate` almacena el número de casos (`cases` en `table1`) y la población total (`population` en `table2`), separados por `/`. Además, como los tipos de variables tambien deben coincidir, con `str()` podemos ver que en `table5` todas las variables son de tipo `character` mientras que en `table1`, todas son numéricas excepto `country`, que si es `character`.

Por tanto, vamos a unir el siglo y año en una única variable llamada `year` y de tipo numérico, como en `table1`. Con `separate()`, obtenemos los casos y la población por separado, indicando el separador que los divide, y finalmente, las convertimos a entero.

```
table5_tidy <- table5 %>%
  unite("year", century, year, sep = "") %>%
  separate(rate, into = c("cases", "population"), sep = "/") %>%
  mutate(
    year = as.integer(year),
    cases = as.integer(cases),
    population = as.integer(population)
  )

head(table5_tidy)
```

```
# A tibble: 6 x 4
  country      year  cases population
  <chr>       <int>  <int>      <int>
1 Afghanistan  1999    745   19987071
2 Afghanistan  2000   2666   20595360
3 Brazil       1999  37737  172006362
4 Brazil       2000  80488  174504898
5 China        1999 212258 1272915272
6 China        2000 213766 1280428583
```

```
str(table5_tidy)
```

```
tibble [6 x 4] (S3: tbl_df/tbl/data.frame)
 $ country   : chr [1:6] "Afghanistan" "Afghanistan" "Brazil" "Brazil" ...
 $ year      : int [1:6] 1999 2000 1999 2000 1999 2000
 $ cases     : int [1:6] 745 2666 37737 80488 212258 213766
 $ population: int [1:6] 19987071 20595360 172006362 174504898 1272915272 1280428583
```

Finalmente, comprobamos si son idénticas.

```
all.equal(table1, table5_tidy)
```

```
[1] TRUE
```

DPLYR

Ejercicio Extra 3

Consigue un data frame denominado `Distance_Manufacturer_model` que contenga 4 columnas (`tailnum`,`Total_distance`, `manufacturer` y `model`) ordenado de mayor a menor en base a la variable `Total_distance`. Para ello une los data frames `Distances_plane` (calculado en el ejercicio anterior y que contiene las distancias totales) y `planes` (que contiene los datos de cada modelo de avión). La variable

que te permitirá unir ambos data frames es `tailnum`. A continuación deberás ordenar, en orden descendente de la variable `Total_distance` el data frame resultante. Finalmente selecciona las variables que tiene que contener la tabla final (`Total_distance`, `manufacturer` y `model`). Utiliza el operador pipe para enlazar las funciones.

Las columnas `manufacturer` y `model` son propias del dataset `planes`, por lo que es necesario hacer un join con `Distances_plane` en el que se mantengan todas sus columnas, y además, añada las dos nuevas. Para ello, utilizamos `left_join()`, incluyendo, de `planes`, solamente aquellos registros donde `tailnum` sea compartido con los registros del otro dataset.

Finalmente, ordenamos por `Total_distance` y, del dataset completo, nos quedamos únicamente con las variables indicadas.

```
Distance_Manufacturer_model <- Distances_plane %>%
  left_join(planes, by = "tailnum") %>%
  arrange(desc(Total_distance)) %>%
  select(tailnum, Total_distance, manufacturer, model)

head(Distance_Manufacturer_model)
```

```
# A tibble: 6 x 4
  tailnum Total_distance manufacturer model
  <chr>            <dbl> <chr>        <chr>
1 N718TW          775381 BOEING       757-231
2 N722TW          764177 BOEING       757-231
3 N721TW          756030 BOEING       757-231
4 N502UA          726276 BOEING       757-222
5 N624AG          718799 BOEING       757-2Q8
6 N505UA          716931 BOEING       757-222
```

Ejercicio Extra 4

Queremos averiguar si existe algún tipo de relación entre el retraso de los vuelos (llegada y salida) y el fabricante. Para ello queremos calcular el retraso medio en la llegada y en la salida de los vuelos (cuando se disponga de esos datos) para cada uno de los fabricantes (`manufacturer`). Propón la estrategia (y el código) a emplear para conseguir proporcionar un data frame ordenado del retraso en la llegada y la salida para cada uno de los fabricantes. ¿Cuáles son los dos fabricantes de aviones que más se retrasan en las salidas? ¿Y en las llegadas?

```
# ##EXTRA incluye IATA code
#
#
# #library(readr)
# # IATA_code <- read_delim("data/IATA_code.csv",delim = "\t", escape_double = FALSE, trim_ws = TRUE)%>
# #   select("IATA code","IATA airlines")
# # names(IATA_code)<-c('code','airline')
# #
# # IATA_code<-IATA_code%>%tidyr::separate(col=code,into=c('borrar','carrier'),sep=' +[-] +')%>%select(
# library(readxl)
# IATA_ICAO <- read_excel("data/Bansard-airlines-codes-IATA-ICAO.xlsx")%>%select(`Airline Name`,`IATA D
#
# flights<-left_join(flights,IATA_ICAO,by='carrier')
# flights%>%filter(!is.na(dep_delay),!is.na(arr_delay))%>%left_join(planes,by='tailnum')%>%group_by(car
```

Ejercicio Extra 5

Lo que hemos hecho en el ejercicio anterior está bien, pero vale más una imagen que mil palabras. Vamos a plantear representaciones gráficas para visualizar el retraso de cada compañía.

- Representa los boxplots de las variables relacionadas con el retraso en la llegada y en la salida (las que terminan con la palabra `delay`) agrupadas por compañía aérea (variable `carrier`) dibujando en dos colores diferentes cada tipo de retraso: tendrás que modificar el data frame con la función `pivot_longer` para crear una nueva variable que indique el tipo de retraso (llegada y salida) que sirva para colorear agrupando por cada tipo de retraso.
- Representa un gráfico de barras que contenga, para cada compañía aérea (`carrier`), el retraso medio de llegadas, de salidas y el retraso global (nueva variable con el promedio del retraso de salida y el retraso de llegadas). El eje x representará la compañía aérea y el eje y el retraso. Las barras de cada tipo de retraso (llegadas, salidas y global) tendrán un color de relleno diferente (tendrás que modificar el data frame con la función `pivot_longer` para crear una nueva variable que indique el tipo de retraso) y se posicionarán una junto a la otra (no apiladas). No olvides añadir etiquetas en los ejes x e y.

¿Cuál de las dos representaciones anteriores es más adecuada para visualizar los retrasos?

Ejercicio Extra 6

Utilizando los conjuntos de datos `flights` y `planes` responde a las siguientes preguntas

1. ¿Qué avión(tailnum) ha realizado más vuelos entre la ruta JFK (origen) y SFO (destino), en relación a su fecha de construcción `year`)? ¿de qué compañia aérea(carrier) es? ¿Cuál es el modelo de avión y cuántos asientos tiene?
2. Indica cuántas compañias aéreas (carrier) operan la ruta entre Nueva York (JFK) y Los Ángeles (LAX). ¿Cuántos aviones distintos han realizado esta ruta ?. Realiza un histograma de las duraciones de los vuelos entre ambos aeropuertos. ¿Son todos vuelos directos ?

Del dataset original, nos quedamos solamente con aquellos vuelos que tengan origen `JFK` y destino `LAX`. A partir de este, con `n_distinct` obtenemos las compañias y aviones que han realizado esta ruta.

```
# 2
JFK_LAX <- flights %>%
  filter(origin == "JFK", dest == "LAX") %>%
  summarise(
    Num_companias = n_distinct(carrier),
    Num_aviones = n_distinct(tailnum, na.rm = TRUE)
  )

JFK_LAX
```

```
  Num_companias Num_aviones
1             5         254
```

Obtenemos **5 compañias** distintas y **254 aviones distintos**.

3. ¿Qué compañía aérea elegirías si tienes que ir desde La Guardia (LGA) a Denver (DEN) en el menor tiempo posible (en promedio), considerando los retrasos de salida y llegada. ¿Cuál es la duración media del vuelo (en horas)?